
Compact FashionMNIST Retrieval and Compression

Artificial Neural Network – Final Project Report

Alperen Davran

University of Antwerp

Lecturers: Arian Sabaghi and José Antonio Oramas Mogrovejo

GitHub: <https://github.com/alperendavran/ann-fashion-retrieval-compression>

June 2026

Abstract

I compare StandardCNN and depthwise separable CNN on FashionMNIST, train a retrieval embedding model from the selected backbone, and test pruning and distillation for a smaller deployment model.

1 Protocol

I used the shared loaders in `src/data.py`, seed 42, and the fixed scaffold in `src/models.py`. I did not run a large hyperparameter search. I use 12,000 training and 2,000 validation examples for Part A model selection, then report classifier test metrics on the full 10,000 FashionMNIST test split. For retrieval, I train on 10,000 fixed metric-training examples and report the final retrieval numbers on a fixed 2,000-example FashionMNIST test subset. A separate 2,000-example validation slice is used only for checkpoint/epoch selection, so the reported retrieval test set is not used for model selection. Retrieval is evaluated with L2-normalized embeddings and pairwise L2 distance. For Recall@1, a query is counted as correct if its nearest non-self neighbor has the same FashionMNIST class. Since FashionMNIST has 10 balanced classes, a random same-class top-1 retrieval baseline is about 10%. FLOPs are counted consistently for Conv2d and Linear layers, so I use them as comparable scaffold-level estimates rather than measured latency.

Stage	Main question	Selection rule
Part A	Which fixed classifier backbone is the best starting point?	Validation macro F1, with parameters and FLOPs also reported.
Part B	Does the selected classifier become a useful retrieval embedding model?	Recall@1 on the retrieval validation split, then final test reporting.
Part C	Which compressed model is most useful for deployment?	Preserve retrieval Recall@1 while reducing dense FLOPs and parameters.

Table 1: Project decision flow. Each stage uses the output of the previous stage.

2 Part A: Backbone Comparison

Part A compares only the two required backbones: **StandardCNN** and **SeparableCNN**. I implemented the missing depthwise separable blocks as depthwise 3×3 convolution followed by pointwise 1×1 convolution, following the idea used in Xception-style models [1]. I did not change the number of blocks, channel sizes, pooling pattern, or classifier width.

Model	LR	Val Acc.	Val F1	Test Acc.	Test F1	Params	FLOPs
StandardCNN	0.0010	0.7865	0.7772	0.7938	0.7874	93,962	14,948,746
StandardCNN	0.0005	0.7430	0.7258	0.7563	0.7410	93,962	14,948,746
SeparableCNN	0.0010	0.8440	0.8404	0.8305	0.8278	31,338	3,972,746
SeparableCNN	0.0005	0.8270	0.8231	0.8139	0.8134	31,338	3,972,746

Table 2: Part A classifier results. Macro F1 is the main selection metric.

I keep **SeparableCNN**. It has the best validation macro F1 and is also much cheaper: about $3.76\times$ fewer FLOPs and $3.00\times$ fewer parameters than the best StandardCNN row.

3 Part B: Retrieval Embeddings

For Part B, I reuse the selected Part A **SeparableCNN** checkpoint and replace the classification head with a 64-dimensional embedding head. I use triplet loss because the objective directly matches retrieval: an anchor should be closer to a positive item than to a negative item [5]. I also ran the allowed contrastive alternative as a control under the same protocol [2].

Model	Best epoch	Recall@1	Recall@5	Recall@10	mAP
Part A initialized embedding	0	0.8160	0.9410	0.9665	0.6968
Contrastive control	1	0.8190	0.9420	0.9670	0.7054
Triplet training, batch-hard	2	0.8295	0.9465	0.9700	0.6918

Table 3: Part B retrieval results. The triplet model is selected because Recall@1 is the main retrieval metric and it reaches the best Recall@1.

The retrieval model learns meaningful similarity, but not perfectly. Triplet training improves Recall@1 from 0.8160 to 0.8295 and also improves Recall@5 and Recall@10. mAP is slightly lower than the initialized embedding, so top-K retrieval improves more than the full ranking average.

I trained both metric models for 8 epochs and selected checkpoints on validation Recall@1. The best epoch is early (triplet: 2, contrastive: 1). The gain over the Part A initialization is small because the backbone already gives strong features on the fixed 10,000-example metric-training subset.

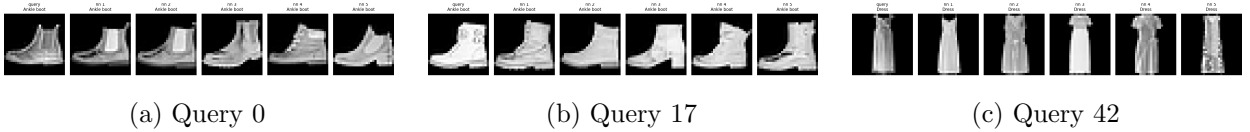


Figure 1: Qualitative retrieval examples. Each panel shows a query followed by nearest neighbors.

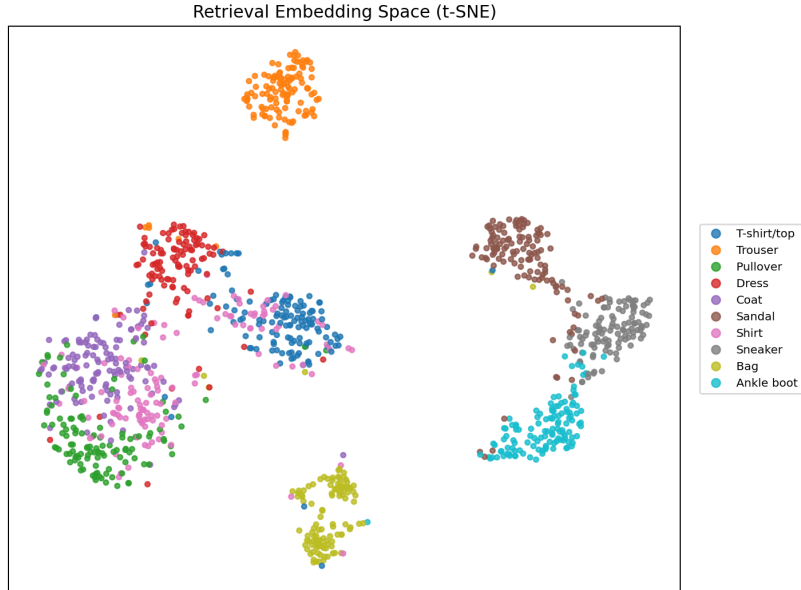


Figure 2: t-SNE visualization of 1,200 retrieval embeddings, using perplexity 30 and seed 42.

The class-level results also explain where retrieval is weaker. Bags, trousers, sandals, sneakers, and ankle boots retrieve well, while Shirt is the hardest class ($\text{Recall@1} = 0.4975$). This is expected because FashionMNIST upper-garment classes such as Shirt, T-shirt/top, Pullover, and Coat are visually close. The full top-1 retrieval confusion matrix is saved with the submitted figures.

4 Part C: Compression

Part C keeps the Part B retrieval teacher and compares the two required compression methods: pruning and knowledge distillation. For pruning, I use 40% global unstructured L1 pruning because it is a simple magnitude-based baseline and directly tests whether small weights can be removed [3]. For distillation, I train the fixed `CompactSeparableCNN` student with relational knowledge distillation (RKD), because the teacher produces embeddings rather than class logits; matching pairwise distances and angles is more relevant for retrieval than matching class probabilities [4].

Model	Recall@1	Preserved	Params	Nonzero	FLOPs
Teacher retrieval model	0.8295	100.0%	38,304	38,304	3,986,624
40% global L1 pruned teacher	0.7495	90.4%	38,304	23,456	3,986,624
RKD compact student	0.7890	95.1%	12,528	12,528	1,190,528

Table 4: Part C compression results.

Pruning keeps 90.4% of teacher Recall@1, but dense FLOPs stay the same because the model is still dense. It would only help on hardware with sparse support. I used unstructured L1 pruning; structured pruning could reduce FLOPs directly, but usually costs more accuracy at the same sparsity.

The distilled student is the better deployment choice here. It keeps 95.1% of teacher Recall@1 while reducing parameters from 38,304 to 12,528 and dense FLOPs from 3,986,624 to 1,190,528.

5 Report Questions

Which backbone would I keep after comparing CNN and separable CNN, and why? I would keep `SeparableCNN`. It has the highest validation macro F1 in Table 2, and it is also much smaller and cheaper than `StandardCNN`. This makes it the best Part A checkpoint to reuse in Part B.

Did the retrieval model learn meaningful similarity? Yes. Recall@1 reaches 0.8295, well above the roughly 10% random same-class baseline, and Recall@1/5/10 improve over the initialized embedding. mAP is slightly lower, and Shirt is the hardest class because upper-garment items are visually similar.

How much performance was preserved after pruning and distillation? The 40% pruned teacher preserves 90.4% of the teacher Recall@1. The RKD compact student preserves 95.1%. So both compression methods keep useful retrieval behavior, but distillation preserves more quality and also reduces dense FLOPs.

What final model would I deploy, and why? I would deploy the RKD compact student. It is smaller, has $3.35\times$ fewer dense FLOPs than the teacher, and preserves more Recall@1 than the pruned model. I would not choose the pruned model as the default limited-hardware deployment unless sparse operations are supported, because its dense FLOPs are unchanged.

6 Run Instructions

The full pipeline can be rerun with:

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install -r requirements.txt
python src/run_project.py
```

The main outputs are stored in `outputs/part_a/`, `outputs/part_b/`, `outputs/part_c/`, and `outputs/figures/`. Local folders such as `.venv/`, `data/`, and `__pycache__/` are not part of the submission.

References

- [1] Francois Chollet. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.
- [2] Raia Hadsell, Sumit Chopra, and Yann LeCun. Dimensionality reduction by learning an invariant mapping. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2006.
- [3] Song Han, Jeff Pool, John Tran, and William J. Dally. Learning both weights and connections for efficient neural networks. In *Advances in Neural Information Processing Systems*, 2015.
- [4] Wonpyo Park, Dongju Kim, Yan Lu, and Minsu Cho. Relational knowledge distillation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019.
- [5] Florian Schroff, Dmitry Kalenichenko, and James Philbin. Facenet: A unified embedding for face recognition and clustering. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015.